Operación Marea Alta

Modelo relacional: tablas, claves primarias (PK) y claves foráneas (FK).

Cómo pasamos del modelo conceptual a las tablas

- Cada entidad del diagrama es una tabla. Los nombres van en singular y sin tildes (Condición pasa a CONDICION, Donación a DONACION).
- En una relación 1 a N, la clave foránea va en la tabla del lado N. Por ejemplo, HOGAR guarda **id_ubicacion** porque muchos hogares tienen una misma ubicación.
- Una relación N a M sin datos propios se vuelve una tabla intermedia con las dos claves. Pasa con **Tiene** (Hogar y Condición, tabla HOGAR_CONDICION) y con **Integra** (Usuario y Equipo, tabla USUARIO_EQUIPO).
- Los tipos de dato están en SQL estándar. **PK** es clave primaria y **FK** es clave foránea, con la tabla a la que apunta.

En total son 20 tablas: las 18 entidades del diagrama más las 2 intermedias.

Tablas

1. UBICACION

Barrios, corregimientos y veredas de Buenaventura donde viven los hogares y trabajan los equipos.

```
UBICACION (
  id_ubicacion   INTEGER       PK,
  zona           VARCHAR(10)   NOT NULL  CHECK (zona IN ('URBANA','RURAL')),
  division       VARCHAR(15)   NOT NULL
                 CHECK (division IN ('BARRIO','CORREGIMIENTO','VEREDA')),
  nombre         VARCHAR(100)  NOT NULL,
  referencia     VARCHAR(200)  NULL,
  UNIQUE (nombre, division),
  CHECK ((zona = 'URBANA' AND division = 'BARRIO')
      OR (zona = 'RURAL'  AND division IN ('CORREGIMIENTO','VEREDA')))
)
```

2. HOGAR

La familia que pide ayuda, con su persona de contacto y cuántas personas viven ahí.

```
HOGAR (
  id_hogar           INTEGER       PK,
  nombre_contacto    VARCHAR(100)  NOT NULL,
  telefono_contacto  VARCHAR(20)   NOT NULL,
  numero_personas    INTEGER       NOT NULL  CHECK (numero_personas > 0),
  id_ubicacion       INTEGER       NOT NULL  FK -> UBICACION(id_ubicacion)
)
INDEX idx_hogar_posible_duplicado ON HOGAR (telefono_contacto, id_ubicacion)  -- no es UNIQUE
```

3. CONDICION

Situaciones que cambian la prioridad: niños pequeños, personas mayores, gestantes, movilidad reducida.

```
CONDICION (
  id_condicion   INTEGER       PK,
  nombre         VARCHAR(60)   NOT NULL  UNIQUE
)
```

4. HOGAR_CONDICION (viene de la relación Tiene)

Dice qué condiciones tiene cada hogar. Un hogar puede tener varias y una condición aplica a varios hogares.

```
HOGAR_CONDICION (
  id_hogar       INTEGER  NOT NULL  FK -> HOGAR(id_hogar),
  id_condicion   INTEGER  NOT NULL  FK -> CONDICION(id_condicion),
  PRIMARY KEY (id_hogar, id_condicion)
)
```

5. NECESIDAD

Los tipos de cosas que se pueden pedir: agua, alimentos, aseo, etc.

```
NECESIDAD (
  id_necesidad      INTEGER      PK,
  nombre_categoria  VARCHAR(60)  NOT NULL  UNIQUE
)
```

6. REPORTE

Cuando un hogar dice qué necesita. Guarda la urgencia, la cantidad estimada y el estado actual.

```
REPORTE (
  id_reporte         INTEGER        PK,
  urgencia           VARCHAR(10)    NOT NULL  CHECK (urgencia IN ('ALTA','MEDIA','BAJA')),
  cantidad_estimada  DECIMAL(10,2)  NULL      CHECK (cantidad_estimada > 0),
  estado             VARCHAR(15)    NOT NULL  DEFAULT 'PENDIENTE'
                     CHECK (estado IN ('PENDIENTE','EN_ATENCION','ATENDIDO')),
  observacion        VARCHAR(300)   NULL,
  fecha              DATE           NOT NULL  DEFAULT CURRENT_DATE,
  id_hogar           INTEGER        NOT NULL  FK -> HOGAR(id_hogar),
  id_necesidad       INTEGER        NOT NULL  FK -> NECESIDAD(id_necesidad)
)
UNIQUE INDEX uq_reporte_abierto ON REPORTE (id_hogar, id_necesidad)
  WHERE estado <> 'ATENDIDO'     -- un solo reporte abierto por hogar y necesidad
```

7. SEGUIMIENTO

Historial de un reporte: cada cambio de estado queda anotado para no perder cómo fue avanzando.

```
SEGUIMIENTO (
  id_seguimiento  INTEGER       PK,
  estado          VARCHAR(15)   NOT NULL
                  CHECK (estado IN ('PENDIENTE','EN_ATENCION','ATENDIDO')),
  fecha           TIMESTAMP     NOT NULL  DEFAULT CURRENT_TIMESTAMP,
  observacion     VARCHAR(300)  NULL,
  id_reporte      INTEGER       NOT NULL  FK -> REPORTE(id_reporte)
)
```

8. DONANTE

Quien da la ayuda: persona, organización comunitaria, empresa o entidad pública.

```
DONANTE (
  id_donante    INTEGER       PK,
  nombre        VARCHAR(100)  NOT NULL,
  tipo_donante  VARCHAR(25)   NOT NULL
                CHECK (tipo_donante IN ('PERSONA','ORGANIZACION_COMUNITARIA',
                                        'EMPRESA','ENTIDAD_PUBLICA')),
  telefono      VARCHAR(20)   NULL
)
```

9. DONACION

Cada vez que un donante entrega algo. Guarda la fecha en que se recibió.

```
DONACION (
  id_donacion      INTEGER  PK,
  fecha_recepcion  DATE     NOT NULL,
  id_donante       INTEGER  NOT NULL  FK -> DONANTE(id_donante)
)
```

10. TIPO_AYUDA

Catálogo de lo que se puede dar, con su unidad de medida estándar.

```
TIPO_AYUDA (
  id_tipo_ayuda  INTEGER      PK,
  nombre         VARCHAR(80)  NOT NULL  UNIQUE,
  unidad_medida  VARCHAR(20)  NOT NULL      -- litros, libras, unidades...
)
```

11. DETALLE_DONACION

Qué trajo y cuánto trajo cada donación. Una donación puede incluir varios tipos de ayuda.

```
DETALLE_DONACION (
  id_detalle_donacion  INTEGER        PK,
  cantidad_recibida    DECIMAL(10,2)  NOT NULL  CHECK (cantidad_recibida > 0),
  id_donacion          INTEGER        NOT NULL  FK -> DONACION(id_donacion),
  id_tipo_ayuda        INTEGER        NOT NULL  FK -> TIPO_AYUDA(id_tipo_ayuda),
  UNIQUE (id_donacion, id_tipo_ayuda)
)
```

12. PUNTO_ATENCION

Los lugares donde se entrega la ayuda.

```
PUNTO_ATENCION (
  id_punto           INTEGER       PK,
  nombre             VARCHAR(100)  NOT NULL  UNIQUE,
  direccion          VARCHAR(150)  NOT NULL,
  estado_habilitado  BOOLEAN       NOT NULL  DEFAULT TRUE
)
```

13. USUARIO

La persona del equipo que usa el sistema, por ejemplo quien registra las entregas.

```
USUARIO (
  id_usuario  INTEGER       PK,
  nombre      VARCHAR(100)  NOT NULL,
  telefono    VARCHAR(20)   NULL,
  estado      VARCHAR(10)   NOT NULL  DEFAULT 'ACTIVO'
              CHECK (estado IN ('ACTIVO','INACTIVO'))
)
```

14. FUNCION

Los roles que se pueden tener: registrador, coordinador, encargado de punto, etc.

```
FUNCION (
  id_funcion   INTEGER       PK,
  nombre       VARCHAR(50)   NOT NULL  UNIQUE,
  descripcion  VARCHAR(200)  NULL
)
```

15. USUARIO_FUNCION

Qué función tiene cada usuario y desde cuándo hasta cuándo. Permite quitar un rol sin borrar a la persona.

```
USUARIO_FUNCION (
  id_usuario_funcion  INTEGER      PK,
  fecha_inicio        DATE         NOT NULL,
  fecha_fin           DATE         NULL,
  estado              VARCHAR(10)  NOT NULL  DEFAULT 'ACTIVA'
                      CHECK (estado IN ('ACTIVA','INACTIVA')),
  id_usuario          INTEGER      NOT NULL  FK -> USUARIO(id_usuario),
  id_funcion          INTEGER      NOT NULL  FK -> FUNCION(id_funcion),
  UNIQUE (id_usuario, id_funcion, fecha_inicio),
  CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio)
)
```

16. EQUIPO

Un grupo de usuarios que sale a trabajar junto, para atender varios barrios al mismo tiempo.

```
EQUIPO (
  id_equipo  INTEGER      PK,
  nombre     VARCHAR(80)  NOT NULL  UNIQUE
)
```

17. USUARIO_EQUIPO (viene de la relación Integra)

Dice qué usuarios hacen parte de cada equipo.

```
USUARIO_EQUIPO (
  id_usuario  INTEGER  NOT NULL  FK -> USUARIO(id_usuario),
  id_equipo   INTEGER  NOT NULL  FK -> EQUIPO(id_equipo),
  PRIMARY KEY (id_usuario, id_equipo)
)
```

18. JORNADA

Cada salida de un equipo a un barrio, con su fecha y estado.

```
JORNADA (
  id_jornada    INTEGER      PK,
  fecha         DATE         NOT NULL,
  estado        VARCHAR(12)  NOT NULL  DEFAULT 'PROGRAMADA'
                CHECK (estado IN ('PROGRAMADA','EN_CURSO','FINALIZADA')),
  id_equipo     INTEGER      NOT NULL  FK -> EQUIPO(id_equipo),
  id_ubicacion  INTEGER      NOT NULL  FK -> UBICACION(id_ubicacion),
  UNIQUE (id_equipo, fecha)      -- un equipo no puede estar en dos barrios el mismo dia
)
```

19. ENTREGA

Cada vez que se le da ayuda a un hogar. Guarda las dos fechas, el estado y quién dejó la constancia.

```
ENTREGA (
  id_entrega      INTEGER      PK,
  fecha_entrega   DATE         NOT NULL,
  fecha_registro  TIMESTAMP    NOT NULL  DEFAULT CURRENT_TIMESTAMP,
  estado          VARCHAR(12)  NOT NULL  DEFAULT 'PENDIENTE'
                  CHECK (estado IN ('PENDIENTE','CONFIRMADA')),
  id_hogar        INTEGER      NOT NULL  FK -> HOGAR(id_hogar),
  id_punto        INTEGER      NOT NULL  FK -> PUNTO_ATENCION(id_punto),
  id_usuario      INTEGER      NOT NULL  FK -> USUARIO(id_usuario),
  id_jornada      INTEGER      NULL      FK -> JORNADA(id_jornada),
  CHECK (CAST(fecha_registro AS DATE) >= fecha_entrega)
)
```

20. DETALLE_ENTREGA

Qué se le dio al hogar y cuánto en esa entrega. Con esto se calcula lo que queda disponible.

```
DETALLE_ENTREGA (
  id_detalle_entrega  INTEGER        PK,
  cantidad_entregada  DECIMAL(10,2)  NOT NULL  CHECK (cantidad_entregada > 0),
  id_entrega          INTEGER        NOT NULL  FK -> ENTREGA(id_entrega),
  id_tipo_ayuda       INTEGER        NOT NULL  FK -> TIPO_AYUDA(id_tipo_ayuda),
  UNIQUE (id_entrega, id_tipo_ayuda)
)
```

Cómo se calcula lo disponible

No hay una tabla ni un campo de inventario. Lo disponible de cada tipo de ayuda es lo recibido menos lo entregado en entregas confirmadas:

```
SELECT t.id_tipo_ayuda, t.nombre, t.unidad_medida,
       COALESCE((SELECT SUM(dd.cantidad_recibida)
                 FROM DETALLE_DONACION dd
                 WHERE dd.id_tipo_ayuda = t.id_tipo_ayuda), 0)
     - COALESCE((SELECT SUM(de.cantidad_entregada)
                 FROM DETALLE_ENTREGA de
                 JOIN ENTREGA e ON e.id_entrega = de.id_entrega
                 WHERE de.id_tipo_ayuda = t.id_tipo_ayuda
                   AND e.estado = 'CONFIRMADA'), 0) AS disponible
FROM TIPO_AYUDA t;
```

Reglas y cómo las controlamos

Regla	Cómo se controla
Las cantidades siempre son mayores que cero.	CHECK en cantidad_estimada, cantidad_recibida y cantidad_entregada.
La fecha de registro no puede ser anterior a la de entrega, pero puede ser el mismo día.	CHECK con >= en ENTREGA.
No se puede entregar más de lo que hay.	Al confirmar una entrega se hace una transacción: se revisan las existencias con la consulta de arriba y, si no alcanza, se hace ROLLBACK y se rechaza la entrega completa.
Un hogar no puede tener dos reportes abiertos de la misma necesidad.	Índice único parcial sobre (id_hogar, id_necesidad) mientras el estado no sea ATENDIDO.
Evitar hogares duplicados.	Índice normal (no único) en (telefono_contacto, id_ubicacion). La aplicación avisa cuando hay coincidencia y la persona confirma. No se exige nombre único porque se escribe distinto.
Una función no puede terminar antes de empezar.	CHECK en USUARIO_FUNCION.
Un equipo no puede tener dos jornadas el mismo día.	UNIQUE (id_equipo, fecha) en JORNADA.
Un equipo debe tener al menos un usuario.	Lo revisa la aplicación al crear el equipo, porque las claves solas no lo obligan.
Cada cambio de estado de un reporte debe quedar anotado.	Al cambiar REPORTE.estado, la aplicación inserta una fila en SEGUIMIENTO en la misma transacción.